

Retrieval Engine Competition Project

Project Report

Group: SeaFour

Maksym DOLHOV

Mehdi AGHAEI

Nguyen Ho Bao KHANH

Project Scope

This report documents the notebook-based submission workflow for the Kaggle retrieval competition. The implementation lives in `notebooks/kaggle-submission.ipynb`, which loads the competition data, preprocesses text, runs four retrieval methods (TF-IDF, BM25+, Embedding Search, and a Hybrid BM25+ + Embedding re-ranking approach), evaluates them on the training set, and writes the final submission CSV with the best model.

Main workflow	<code>notebooks/kaggle-submission.ipynb</code>
Submission file	<code>notebooks/solutions_SeaFour.csv</code>
Report	<code>report/retrieval_project_report.pdf</code>
Date	March 1, 2026

Contents

1 Project At A Glance

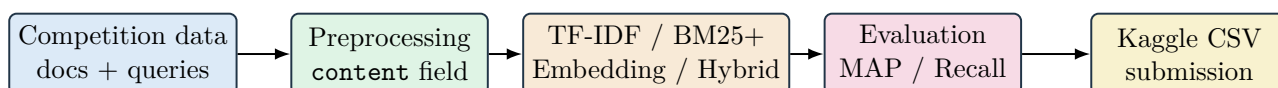
The submission workflow is notebook-first. One Kaggle notebook contains all the logic needed to reproduce the submission file:

- `notebooks/kaggle-submission.ipynb` — active implementation.
- `notebooks/solutions_SeaFour.csv` — generated Kaggle-format output.
- `report/retrieval_project_report.pdf` — this report.

1.1 Repository Layout

Path	Purpose
<code>data/</code>	Competition files: documents, queries, ground-truth, and sample submission.
<code>notebooks/</code>	Active notebook and generated CSV.
<code>report/</code>	LaTeX source and compiled PDF.
<code>requirements.txt</code>	Python dependencies.

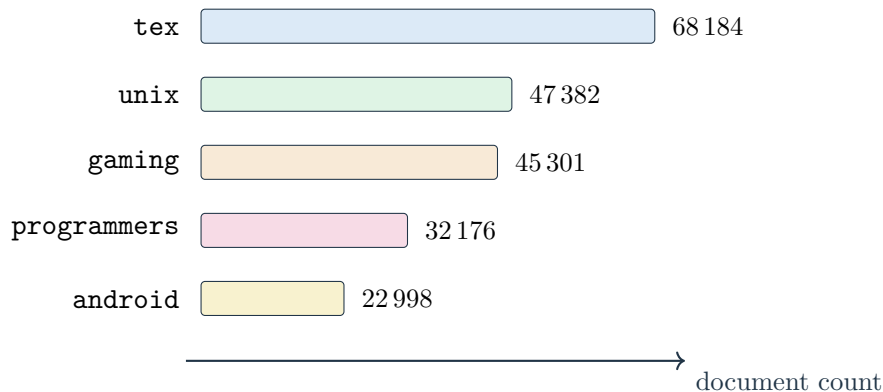
1.2 Pipeline Overview



2 Dataset

	Count	File
Documents	216,041	<code>docs.json</code>
Training queries	327	<code>queries_train.json</code>
Test queries	141	<code>queries_test.json</code>
Training qrels	327	<code>qgts_train.json</code>

Each document has fields `id`, `text`, `title`, `tags` (list), and `category`. The five categories and their document counts:



The ground-truth file maps each training query to a set of relevant document IDs with binary relevance labels.

3 Preprocessing

All retrieval methods receive the same normalised input. A single `content` column is built from the raw fields:

- **Documents:** title + text + tags (tags are space-joined).
- **Queries:** title + text.

Every value is lowercased. Missing values (`None`, `NaN`), lists, and tuples are converted to plain strings by `value_to_text()`.

```
def value_to_text(value):
    if value is None:
        return ''
    if isinstance(value, (list, tuple)):
        return ' '.join(str(v) for v in value)
    if pd.isna(value):
        return ''
    return str(value)

def create_content_column(df, columns):
    out = df.copy()
    for col in columns:
        if col not in out.columns:
            out[col] = ''
    merged = []
    for _, row in out[columns].iterrows():
        text = ' '.join(
            value_to_text(row[col]) for col in columns
        ).strip().lower()
        merged.append(text)
    out['content'] = merged
    out['id'] = out['id'].astype(str)
    return out
```

Why a single content field?

Using one unified text column ensures that TF-IDF, BM25+, and the embedding model all receive the same textual representation. This eliminates inconsistencies in feature construction across methods.

4 Retrieval Methods

The notebook implements four retrieval methods, covering sparse lexical, dense semantic, and hybrid approaches.

4.1 Method 1: TF-IDF (Sparse Lexical Baseline)

TF-IDF (Term Frequency–Inverse Document Frequency) represents each document and query as a sparse vector of weighted term counts, then ranks documents by cosine similarity.

Configuration. The notebook uses `TfidfVectorizer` with:

- Unigrams and bigrams (`ngram_range=(1,2)`).

- `min_df=2` to prune extremely rare terms (fallback to 1 for small subsets).
- Lowercase normalisation.

Scoring.

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{N}{\text{df}(t)}\right)$$

Documents are ranked by cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

```
vectorizer = TfidfVectorizer(
    lowercase=True, ngram_range=(1, 2), min_df=2
)
doc_vectors = vectorizer.fit_transform(docs_df['content'])
query_vectors = vectorizer.transform(queries_df['content'])
scores = cosine_similarity(query_vectors, doc_vectors)
```

Strengths

Fast to compute on large corpora; bigrams preserve short technical expressions; easy to interpret.

4.2 Method 2: BM25+ (Probabilistic Lexical Model)

BM25+ is a probabilistic ranking function that extends TF saturation with document-length normalisation and a positive lower bound (δ).

Tokenisation. A custom tokeniser normalises separators (`-`, `_`, `/` $\rightarrow$ space) and extracts alphanumeric tokens via the regex `[a-z0-9]+`.

```
_TOKEN_RE = re.compile(r'[a-z0-9]+')

def tokenize(text):
    txt = str(text or '').lower()
    txt = re.sub(r'[-_ /]', ' ', txt)
    return _TOKEN_RE.findall(txt)
```

Scoring.

$$\text{BM25+}(q, d) = \sum_{t \in q} \text{IDF}(t) \left(\frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)} + \delta \right)$$

The implementation uses `BM25Plus` from the `rank-bm25` library with default parameters ($k_1 = 1.5$, $b = 0.75$, $\delta = 1$).

```
from rank_bm25 import BM25Plus
tokenized_corpus = [tokenize(t) for t in docs_df['content']]
bm25 = BM25Plus(tokenized_corpus)
scores = bm25.get_scores(tokenize(query_text))
```

Strengths

Robust lexical ranker for technical text; handles term frequency saturation and document-length effects; fast inference without GPU.

4.3 Method 3: Embedding Search (Dense Semantic Retrieval)

Embedding search encodes documents and queries into dense vectors using a pre-trained Sentence-Transformer, then ranks by cosine similarity in the embedding space.

Model. The notebook uses `all-MiniLM-L6-v2`, a lightweight model that produces 384-dimensional L2-normalised embeddings. It captures semantic meaning beyond exact keyword matches.

Scoring. Since the embeddings are L2-normalised, cosine similarity reduces to a dot product:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \mathbf{q}^\top \mathbf{d} \quad (\|\mathbf{q}\| = \|\mathbf{d}\| = 1)$$

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

doc_embeddings = model.encode(
    docs_df['content'].tolist(),
    normalize_embeddings=True,
)
query_embeddings = model.encode(
    queries_df['content'].tolist(),
    normalize_embeddings=True,
)
# dot product = cosine similarity (L2-normalised)
scores = query_embeddings @ doc_embeddings.T
```

Strengths

Captures semantic similarity (paraphrases, synonyms); works even when query and document share no common terms. The trade-off is higher computational cost for encoding 216k documents.

4.4 Method 4: Hybrid BM25+ → Embedding Re-ranking

The hybrid method combines the strengths of BM25+ (high lexical recall, fast) with embedding search (semantic precision). It operates in three stages:

1. **Stage 1 — BM25+ Candidate Retrieval.** BM25+ scores all 216k documents and selects the top- N candidates per query (default $N = 500$). This is fast and ensures lexical recall.
2. **Stage 2 — Embedding Re-scoring.** Only the N candidate documents are encoded with the Sentence-Transformer. The embedding model computes cosine similarity between each query and its candidates.
3. **Stage 3 — Score Fusion.** BM25+ scores are min-max normalised to $[0, 1]$, then fused with embedding scores using a weighted sum:

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \widehat{\text{BM25+}}(q, d) + (1 - \alpha) \cdot \text{sim}_{\text{emb}}(q, d)$$

where $\alpha \in [0, 1]$ controls the trade-off (default $\alpha = 0.35$). The top- K documents by fused score form the final ranked list.

Key advantage. By encoding only N candidates instead of 216k documents, the hybrid method is much faster than full embedding search while still benefiting from semantic re-ranking. BM25+ provides the recall base; the embedding model improves precision by promoting semantically relevant documents that BM25+ might rank lower.

```

# Stage 1: BM25+ candidates
scores = bm25.get_scores(query_tokens)
top_n = np.argsort(scores)[-N:] [::-1]

# Stage 2: encode only candidates
cand_embs = model.encode(docs[top_n])
emb_scores = query_emb @ cand_embs.T

# Stage 3: fuse
bm25_norm = (bm25_scores - min) / (max - min)
fused = alpha * bm25_norm + (1-alpha) * emb_scores
final_top_k = np.argsort(fused)[-K:] [::-1]

```

Why this works

BM25+ and embeddings have complementary failure modes. BM25+ struggles with paraphrases and synonyms; embeddings can miss exact rare terms. Fusing both yields better overall ranking than either alone.

4.5 Method Comparison

Method	Type	Representation	Key Characteristics
TF-IDF	Sparse lexical	Sparse vectors	Fast; relies on exact term overlap; bi-grams help with short phrases
BM25+	Sparse lexical	Token lists	TF saturation + length normalisation; robust default for technical text
Embedding	Dense semantic	384-d dense vectors	Captures meaning beyond keywords; requires transformer inference
Hybrid	Sparse + Dense	Both	BM25+ recall + embedding precision; re-ranks N candidates

5 Evaluation

All four methods are evaluated on the 327 training queries using the provided ground-truth relevance judgements (`qgts_train.json`).

5.1 Metrics

Mean Average Precision (MAP@K). For each query, Average Precision is computed over the top- K retrieved documents:

$$\text{AP}(q) = \frac{1}{|\mathcal{R}_q|} \sum_{k=1}^K \text{Precision}(k) \cdot \mathbb{I}[d_k \in \mathcal{R}_q]$$

MAP@K is the mean of AP values over all evaluated queries.

Recall@K. The fraction of relevant documents appearing in the top- K results:

$$\text{Recall@K}(q) = \frac{|\{d \in \text{top-}K\} \cap \mathcal{R}_q|}{|\mathcal{R}_q|}$$

5.2 Implementation

```
def mean_average_precision(results, ground_truth, k=100):
    aps = []
    for item in results:
        qid = str(item['query_id'])
        if qid not in ground_truth:
            continue
        relevant = ground_truth[qid]
        hits, score = 0, 0.0
        for rank, doc_id in enumerate(
            item['relevant_docs'][:k], 1):
            if str(doc_id) in relevant:
                hits += 1
                score += hits / rank
        aps.append(score / len(relevant) if relevant else 0.0)
    return np.mean(aps) if aps else 0.0

def recall_at_k(results, ground_truth, k=100):
    recalls = []
    for item in results:
        qid = str(item['query_id'])
        if qid not in ground_truth:
            continue
        relevant = ground_truth[qid]
        retrieved = {str(d) for d in item['relevant_docs'][:k]}
        recalls.append(
            len(relevant & retrieved) / len(relevant)
            if relevant else 0.0
        )
    return np.mean(recalls) if recalls else 0.0
```

5.3 Results

The notebook runs all four models on the training queries and produces a comparison table. The results determine which model is used for the final test submission.

Model	MAP@100	Recall@100
TF-IDF	<i>(from notebook)</i>	<i>(from notebook)</i>
BM25+	<i>(from notebook)</i>	<i>(from notebook)</i>
Embedding	<i>(from notebook)</i>	<i>(from notebook)</i>
Hybrid	<i>(from notebook)</i>	<i>(from notebook)</i>

Note

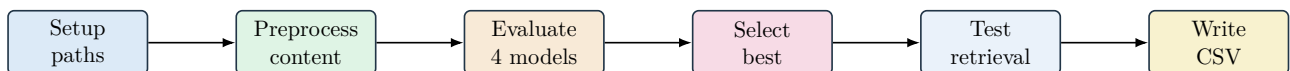
Fill in the actual metric values after running the evaluation cells in the notebook. The `FINAL_MODEL` parameter is then set to the best-performing method.

6 Submission Generation

The final submission is produced by running the selected model on the 141 test queries and writing the output in Kaggle format.

6.1 Notebook Workflow

1. **Environment setup.** Auto-detect the Kaggle input directory; fall back to `../data` for local execution.
2. **Preprocessing.** Build the shared `content` column for documents and queries.
3. **Evaluation.** Run all four models on 327 training queries; compute MAP@100 and Recall@100.
4. **Model selection.** Set `FINAL_MODEL` to the best-performing method (configurable).
5. **Test retrieval.** Run the selected model on 141 test queries.
6. **CSV writing.** Serialise results in Kaggle format.
7. **Preview.** Display the first rows of the submission CSV.



6.2 Configuration

Parameter	Role
<code>FINAL_MODEL</code>	Model for submission: <code>bm25</code> , <code>tfidf</code> , <code>embedding</code> , or <code>hybrid</code> .
<code>TOP_K</code>	Documents per query (100).
<code>OUTPUT_PATH</code>	Output CSV path (<code>solutions_SeaFour.csv</code>).
<code>EMBEDDING_MODEL</code>	Sentence-Transformer model name (<code>all-MiniLM-L6-v2</code>).
<code>EMBEDDING_BATCH</code>	Encoding batch size (256).
<code>HYBRID_BM25_CANDIDATES</code>	BM25+ candidates per query before re-ranking (500).
<code>HYBRID_ALPHA</code>	Weight for BM25+ in score fusion, $\alpha = 0.35$ by default.

6.3 CSV Format

The writer reads the sample submission template, preserves column order, and writes JSON-encoded document ID lists:

```
out_row = {
    id_col: qid,
    pred_col: json.dumps(pred_map[qid]),
}
if category_col is not None:
    out_row[category_col] = row.get(category_col, '?') or '?'
```

7 Dependencies

Package	Purpose
<code>numpy</code>	Array operations and sorting
<code>pandas</code>	DataFrame handling and I/O
<code>scikit-learn</code>	TF-IDF vectoriser and cosine similarity
<code>rank-bm25</code>	BM25+ implementation
<code>sentence-transformers</code>	Pre-trained dense encoders
<code>torch</code>	Backend for sentence-transformers

A Mathematical Details

TF-IDF. Weighs terms by local frequency and global rarity:

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{N}{\text{df}(t)}\right)$$

Cosine Similarity.

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

BM25+.

$$\text{BM25+}(q, d) = \sum_{t \in q} \text{IDF}(t) \left(\frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} + \delta \right)$$

Sentence Embedding Similarity. For L2-normalised vectors:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \mathbf{q}^\top \mathbf{d}$$

Hybrid Score Fusion.

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \frac{\text{BM25+}(q, d) - \min}{\max - \min} + (1 - \alpha) \cdot \text{sim}_{\text{emb}}(q, d)$$

Top-K Selection.

$$\text{TopK}(q) = \text{argsort}(\text{scores}(q))[-K :][::-1]$$